

Module 5: Pandas DataFrame

Author: Lei Wu

Date: 2023-01-01

Version: 1.0.0

Learning Objectives

- Understanding DataFrame data structure
- DataFrame creation and manipulation
- Indexing, selection, and filtering
- Data cleaning and preprocessing
- Basic data analysis operations

Topics Covered

1. Introduction to DataFrames
2. Creating DataFrames from various sources
3. DataFrame attributes and methods
4. Indexing and selection (loc, iloc, at, iat)
5. Filtering and boolean indexing
6. Adding and removing columns/rows
7. Handling missing values
8. Data type conversion
9. Sorting and ranking
10. Basic aggregation functions
11. DataFrame operations and arithmetic

Exercises

- DataFrame creation and manipulation exercises
- Data cleaning and preprocessing tasks

1. Introduction to DataFrames

```
In [ ]: import pandas as pd
import numpy as np

# Create a sample DataFrame
data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'Diana', 'Eve'],
    'Age': [25, 30, 35, 28, 32],
    'City': ['New York', 'Los Angeles', 'Chicago', 'Houston', 'Phoenix'],
    'Salary': [50000, 60000, 70000, 55000, 65000]
```

```

}

df = pd.DataFrame(data)
print("Sample DataFrame:")
print(df)

# DataFrame properties
print(f"\nDataFrame Properties:")
print(f"Shape: {df.shape}")
print(f"Columns: {df.columns.tolist()}")
print(f"Index: {df.index.tolist()}")
print(f>Data types:\n{df.dtypes}")
print(f"Memory usage: {df.memory_usage().sum()} bytes")

# Basic information
print(f"\nDataFrame Info:")
print(df.info())

# Statistical summary
print(f"\nStatistical Summary:")
print(df.describe())

```

3. DataFrame Indexing and Selection

```

In [ ]: # Create a comprehensive DataFrame for indexing examples
data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'Diana', 'Eve', 'Frank', 'Grace',
    'Age': [25, 30, 35, 28, 32, 45, 29, 38],
    'City': ['New York', 'Los Angeles', 'Chicago', 'Houston', 'Phoenix',
    'Salary': [50000, 60000, 70000, 55000, 65000, 80000, 58000, 72000],
    'Department': ['IT', 'HR', 'IT', 'Finance', 'IT', 'Finance', 'HR', 'I
}
df = pd.DataFrame(data, index=['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H'])

print(f"Original DataFrame:\n{df}")

# Column selection
print(f"\n--- Column Selection ---")
print(f"Single column (Name):\n{df['Name']}")
print(f"Multiple columns:\n{df[['Name', 'Age', 'Salary']]}")

# Row selection using iloc (position-based)
print(f"\n--- Row Selection (iloc) ---")
print(f"First row:\n{df.iloc[0]}")
print(f"First 3 rows:\n{df.iloc[:3]}")
print(f"Rows 2 to 5:\n{df.iloc[2:6]}")
print(f>Last 2 rows:\n{df.iloc[-2:]}")

# Row and column selection using iloc
print(f"\n--- Row and Column Selection (iloc) ---")
print(f"First 3 rows, first 3 columns:\n{df.iloc[:3, :3]}")
print(f"Rows 1, 3, 5 and columns 0, 2, 4:\n{df.iloc[[1, 3, 5], [0, 2, 4]]}")

# Label-based selection using loc
print(f"\n--- Label-based Selection (loc) ---")
print(f"Row with index 'C':\n{df.loc['C']}")
print(f"Rows 'A' to 'D':\n{df.loc['A':'D']}")
print(f"Rows 'B', 'D', 'F' and columns 'Name', 'Salary':\n{df.loc[['B', '

```

```

# Boolean indexing
print(f"\n--- Boolean Indexing ---")
print(f"People with age > 30:\n{df[df['Age'] > 30]}")
print(f"People in IT department:\n{df[df['Department'] == 'IT']}")
print(f"People with salary between 50000 and 70000:\n{df[(df['Salary'] >= 50000) & (df['Salary'] <= 70000)]}")

# Using .at and .iat for single value access
print(f"\n--- Single Value Access ---")
print(f"Value at row 0, column 'Name': {df.at[0, 'Name']}")
print(f"Value at position (2, 3): {df.iat[2, 3]}")

# Setting values
df_copy = df.copy()
df_copy.loc['A', 'Salary'] = 55000
df_copy.iloc[1, 2] = 'San Francisco'
print(f"\nAfter modification:\n{df_copy}")

```

4. Data Cleaning and Preprocessing

```

In [ ]: # Create a DataFrame with various data quality issues
data_messy = {
    'Name': ['Alice', 'Bob', 'Charlie', 'Diana', 'Eve', 'Frank', 'Grace',
            'Age': [25, 30, 35, 28, 32, 45, 29, 38, np.nan, 42],
            'City': ['New York', 'Los Angeles', 'Chicago', 'Houston', 'Phoenix',
            'Salary': [50000, 60000, 70000, 55000, 65000, 80000, 58000, 72000, 62
            'Department': ['IT', 'HR', 'IT', 'Finance', 'IT', 'Finance', 'HR', 'I
            'Email': ['alice@company.com', 'bob@company.com', 'charlie@company.co
                    'eve@company.com', 'frank@company.com', 'grace@company.com'
                    'ivy@company.com', 'jack@company.com'],
            'Phone': ['555-1234', '555-5678', '555-9012', '555-3456', '555-7890',
                    '555-2345', '555-6789', '555-0123', '555-4567', '555-8901']
}

# Add some missing values and inconsistencies
df_messy = pd.DataFrame(data_messy)
df_messy.loc[2, 'City'] = 'chicago' # Inconsistent case
df_messy.loc[5, 'Department'] = 'FINANCE' # Inconsistent case
df_messy.loc[7, 'Email'] = np.nan # Missing email
df_messy.loc[8, 'Phone'] = '555-XXXX' # Invalid phone

print(f"Messy DataFrame:\n{df_messy}")

# Check for missing values
print(f"\n--- Missing Values Analysis ---")
print(f"Missing values per column:\n{df_messy.isnull().sum()}")
print(f"Percentage of missing values:\n{(df_messy.isnull().sum() / len(df

# Check for duplicates
print(f"\n--- Duplicate Analysis ---")
print(f"Number of duplicate rows: {df_messy.duplicated().sum()}")
print(f"Duplicate rows:\n{df_messy[df_messy.duplicated()]}")

# Data type analysis
print(f"\n--- Data Type Analysis ---")
print(f>Data types:\n{df_messy.dtypes}")

# Handle missing values

```

```

print(f"\n--- Handling Missing Values ---")
# Fill missing age with median
df_clean = df_messy.copy()
df_clean['Age'] = df_clean['Age'].fillna(df_clean['Age'].median())
print(f"After filling missing age with median:\n{df_clean['Age']}")

# Fill missing email with 'unknown@company.com'
df_clean['Email'] = df_clean['Email'].fillna('unknown@company.com')
print(f"After filling missing email:\n{df_clean['Email']}")

# Standardize text data
print(f"\n--- Text Standardization ---")
df_clean['City'] = df_clean['City'].str.title()
df_clean['Department'] = df_clean['Department'].str.title()
print(f"Standardized City:\n{df_clean['City']}")
print(f"Standardized Department:\n{df_clean['Department']}")

# Data validation
print(f"\n--- Data Validation ---")
# Check email format
email_pattern = r'^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$'
valid_emails = df_clean['Email'].str.match(email_pattern)
print(f"Valid email addresses: {valid_emails.sum()}/{len(df_clean)}")

# Check phone format
phone_pattern = r'^\d{3}-\d{4}$'
valid_phones = df_clean['Phone'].str.match(phone_pattern)
print(f"Valid phone numbers: {valid_phones.sum()}/{len(df_clean)}")

# Create age groups
print(f"\n--- Feature Engineering ---")
df_clean['Age_Group'] = pd.cut(df_clean['Age'],
                               bins=[0, 30, 40, 50, 100],
                               labels=['Young', 'Middle', 'Senior', 'Elder'])
print(f"Age groups:\n{df_clean['Age_Group'].value_counts()}")

# Create salary categories
df_clean['Salary_Category'] = pd.cut(df_clean['Salary'],
                                     bins=[0, 55000, 65000, 75000, 100000],
                                     labels=['Low', 'Medium', 'High', 'Ver'])
print(f"Salary categories:\n{df_clean['Salary_Category'].value_counts()}")

print(f"\nCleaned DataFrame:\n{df_clean}")

```

2. Creating DataFrames from Various Sources

```

In [ ]: # 1. From dictionary
data_dict = {
    'A': [1, 2, 3, 4],
    'B': [5, 6, 7, 8],
    'C': [9, 10, 11, 12]
}
df1 = pd.DataFrame(data_dict)
print("From dictionary:")
print(df1)

# 2. From list of lists
data_list = [[1, 5, 9], [2, 6, 10], [3, 7, 11], [4, 8, 12]]

```

```

df2 = pd.DataFrame(data_list, columns=['A', 'B', 'C'])
print("\nFrom list of lists:")
print(df2)

# 3. From NumPy array
np_array = np.random.rand(4, 3)
df3 = pd.DataFrame(np_array, columns=['X', 'Y', 'Z'])
print("\nFrom NumPy array:")
print(df3)

# 4. From list of dictionaries
data_dicts = [
    {'Name': 'Alice', 'Age': 25, 'City': 'NYC'},
    {'Name': 'Bob', 'Age': 30, 'City': 'LA'},
    {'Name': 'Charlie', 'Age': 35, 'City': 'Chicago'}
]
df4 = pd.DataFrame(data_dicts)
print("\nFrom list of dictionaries:")
print(df4)

# 5. From CSV-like string
csv_string = """Name,Age,City
Alice,25,New York
Bob,30,Los Angeles
Charlie,35,Chicago"""
from io import StringIO
df5 = pd.read_csv(StringIO(csv_string))
print("\nFrom CSV string:")
print(df5)

# 6. Empty DataFrame with specified columns
df6 = pd.DataFrame(columns=['Name', 'Age', 'City'])
print("\nEmpty DataFrame:")
print(df6)

```

3. DataFrame Attributes and Methods

```

In [ ]: # Create a comprehensive DataFrame for demonstrating attributes and methods
data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'Diana', 'Eve', 'Frank', 'Grace',
    'Age': [25, 30, 35, 28, 32, 45, 29, 38],
    'City': ['New York', 'Los Angeles', 'Chicago', 'Houston', 'Phoenix',
    'Salary': [50000, 60000, 70000, 55000, 65000, 80000, 58000, 72000],
    'Department': ['IT', 'HR', 'IT', 'Finance', 'IT', 'Finance', 'HR', 'I
    'Experience': [2, 5, 8, 3, 6, 12, 4, 9]
}
df = pd.DataFrame(data, index=['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H'])

print("Original DataFrame:")
print(df)

# Basic attributes
print(f"\n--- Basic Attributes ---")
print(f"Shape: {df.shape}")
print(f"Size: {df.size}")
print(f"Dimensions: {df.ndim}")
print(f"Index: {df.index}")
print(f"Columns: {df.columns}")

```

```

print(f"Data types:\n{df.dtypes}")
print(f"Memory usage:\n{df.memory_usage()}")

# DataFrame information
print(f"\n--- DataFrame Information ---")
print("Info method:")
print(df.info())

# Statistical methods
print(f"\n--- Statistical Methods ---")
print("Describe method (numeric columns only):")
print(df.describe())

print("\nDescribe method (include all columns):")
print(df.describe(include='all'))

# Value counts for categorical data
print(f"\n--- Value Counts ---")
print("Department value counts:")
print(df['Department'].value_counts())

print("\nCity value counts:")
print(df['City'].value_counts())

# Head and tail methods
print(f"\n--- Head and Tail ---")
print("First 3 rows:")
print(df.head(3))

print("\nLast 3 rows:")
print(df.tail(3))

# Sample method
print(f"\n--- Sampling ---")
print("Random sample of 3 rows:")
print(df.sample(3))

print("\nRandom sample of 50% of data:")
print(df.sample(frac=0.5))

# Transpose
print(f"\n--- Transpose ---")
print("Transposed DataFrame:")
print(df.T)

# Copy method
print(f"\n--- Copy Method ---")
df_copy = df.copy()
df_copy.loc['A', 'Salary'] = 99999
print("Original DataFrame (unchanged):")
print(df['Salary'].head(3))
print("\nCopied DataFrame (modified):")
print(df_copy['Salary'].head(3))

# Check for empty DataFrame
print(f"\n--- Empty Check ---")
print(f"Is DataFrame empty? {df.empty}")
print(f"Number of rows: {len(df)}")

# Check for null values

```

```

print(f"\n--- Null Value Check ---")
print("Null values per column:")
print(df.isnull().sum())
print("\nAny null values? {df.isnull().any().any()}")

# Check for duplicates
print(f"\n--- Duplicate Check ---")
print(f"Number of duplicate rows: {df.duplicated().sum()}")
print(f"Any duplicates? {df.duplicated().any()}")

# Column operations
print(f"\n--- Column Operations ---")
print("Column names as list:")
print(df.columns.tolist())

print("\nColumn data types:")
print(df.dtypes.to_dict())

# Index operations
print(f"\n--- Index Operations ---")
print("Index as list:")
print(df.index.tolist())

print("\nReset index:")
df_reset = df.reset_index()
print(df_reset.head(3))

print("\nSet index to Name:")
df_name_index = df.set_index('Name')
print(df_name_index.head(3))

```

5. Filtering and Boolean Indexing

```

In [ ]: # Create a comprehensive DataFrame for filtering examples
data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'Diana', 'Eve', 'Frank', 'Grace',
    'Age': [25, 30, 35, 28, 32, 45, 29, 38, 26, 42],
    'City': ['New York', 'Los Angeles', 'Chicago', 'Houston', 'Phoenix',
    'Salary': [50000, 60000, 70000, 55000, 65000, 80000, 58000, 72000, 62
    'Department': ['IT', 'HR', 'IT', 'Finance', 'IT', 'Finance', 'HR', 'I
    'Experience': [2, 5, 8, 3, 6, 12, 4, 9, 1, 10],
    'Performance': ['Good', 'Excellent', 'Good', 'Average', 'Excellent',
}
df = pd.DataFrame(data, index=['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'I

print("Original DataFrame:")
print(df)

# Basic boolean indexing
print(f"\n--- Basic Boolean Indexing ---")
print("People with age > 30:")
print(df[df['Age'] > 30])

print("\nPeople in IT department:")
print(df[df['Department'] == 'IT'])

print("\nPeople with salary >= 60000:")
print(df[df['Salary'] >= 60000])

```

```

# Multiple conditions with logical operators
print(f"\n--- Multiple Conditions ---")
print("IT employees with salary > 60000:")
it_high_salary = df[(df['Department'] == 'IT') & (df['Salary'] > 60000)]
print(it_high_salary)

print("\nPeople aged 25-35 OR with salary > 70000:")
age_salary_filter = df[(df['Age'] >= 25) & (df['Age'] <= 35) | (df['Salary'] > 70000)]
print(age_salary_filter)

print("\nPeople in IT OR Finance departments:")
dept_filter = df[(df['Department'] == 'IT') | (df['Department'] == 'Finance')]
print(dept_filter)

# Using isin() for multiple values
print(f"\n--- Using isin() Method ---")
print("People in IT or HR departments:")
print(df[df['Department'].isin(['IT', 'HR'])])

print("\nPeople in major cities:")
major_cities = ['New York', 'Los Angeles', 'Chicago', 'Houston']
print(df[df['City'].isin(major_cities)])

# String filtering
print(f"\n--- String Filtering ---")
print("Names starting with 'A':")
print(df[df['Name'].str.startswith('A')])

print("\nNames containing 'ar':")
print(df[df['Name'].str.contains('ar', case=False)])

print("\nNames ending with 'e':")
print(df[df['Name'].str.endswith('e')])

# Numeric range filtering
print(f"\n--- Numeric Range Filtering ---")
print("Age between 30 and 40 (inclusive):")
print(df[(df['Age'] >= 30) & (df['Age'] <= 40)])

print("\nSalary in top 25% (above 75th percentile):")
salary_75th = df['Salary'].quantile(0.75)
print(f"75th percentile salary: {salary_75th}")
print(df[df['Salary'] > salary_75th])

# Using query() method
print(f"\n--- Using query() Method ---")
print("IT employees with experience > 5 years:")
print(df.query("Department == 'IT' and Experience > 5"))

print("\nEmployees aged 25-35 with salary 50000-70000:")
print(df.query("Age >= 25 and Age <= 35 and Salary >= 50000 and Salary <= 70000"))

# Negation with ~ operator
print(f"\n--- Negation with ~ Operator ---")
print("People NOT in IT department:")
print(df[~df['Department'].isin(['IT'])])

print("\nPeople with salary NOT between 55000 and 65000:")
print(df[~((df['Salary'] >= 55000) & (df['Salary'] <= 65000))])

```



```

# Complex filtering with multiple criteria
print(f"\n--- Complex Filtering ---")
print("Senior employees (age > 35) in IT with excellent performance:")
senior_it_excellent = df[
    (df['Age'] > 35) &
    (df['Department'] == 'IT') &
    (df['Performance'] == 'Excellent')
]
print(senior_it_excellent)

print("\nHigh performers (Excellent or Good) with salary > 60000:")
high_performers = df[
    (df['Performance'].isin(['Excellent', 'Good'])) &
    (df['Salary'] > 60000)
]
print(high_performers)

# Filtering with null values
print(f"\n--- Filtering with Null Values ---")
# Add some null values for demonstration
df_with_nulls = df.copy()
df_with_nulls.loc['A', 'Experience'] = np.nan
df_with_nulls.loc['C', 'Performance'] = np.nan

print("DataFrame with some null values:")
print(df_with_nulls[['Name', 'Experience', 'Performance']])

print("\nRows with non-null Experience:")
print(df_with_nulls[df_with_nulls['Experience'].notna()])

print("\nRows with null Performance:")
print(df_with_nulls[df_with_nulls['Performance'].isna()])

# Using where() method
print(f"\n--- Using where() Method ---")
print("Salary where age > 30, otherwise NaN:")
print(df.where(df['Age'] > 30)['Salary'])

# Using mask() method (opposite of where)
print(f"\n--- Using mask() Method ---")
print("Salary where age <= 30, otherwise NaN:")
print(df.mask(df['Age'] > 30)['Salary'])

```

6. Adding and Removing Columns/Rows

```

In [ ]: # Create a base DataFrame for demonstrating column and row operations
data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'Diana', 'Eve'],
    'Age': [25, 30, 35, 28, 32],
    'City': ['New York', 'Los Angeles', 'Chicago', 'Houston', 'Phoenix'],
    'Salary': [50000, 60000, 70000, 55000, 65000]
}
df = pd.DataFrame(data, index=['A', 'B', 'C', 'D', 'E'])

print("Original DataFrame:")
print(df)

```

```

# Adding columns
print(f"\n--- Adding Columns ---")

# Method 1: Direct assignment
df['Department'] = ['IT', 'HR', 'IT', 'Finance', 'IT']
print("After adding Department column:")
print(df)

# Method 2: Using assign() method
df = df.assign(Experience=[2, 5, 8, 3, 6])
print("\nAfter adding Experience column using assign():")
print(df)

# Method 3: Adding multiple columns at once
df[['Performance', 'Manager']] = [['Good', 'Excellent', 'Good', 'Average',
                                     'John', 'Jane', 'John', 'Mike', 'John']
print("\nAfter adding Performance and Manager columns:")
print(df)

# Method 4: Adding calculated columns
df['Salary_Plus_Bonus'] = df['Salary'] * 1.1 # 10% bonus
df['Age_Group'] = df['Age'].apply(lambda x: 'Young' if x < 30 else 'Senior')
print("\nAfter adding calculated columns:")
print(df)

# Method 5: Adding column with insert()
df.insert(2, 'Employee_ID', ['E001', 'E002', 'E003', 'E004', 'E005'])
print("\nAfter inserting Employee_ID at position 2:")
print(df)

# Removing columns
print(f"\n--- Removing Columns ---")

# Method 1: Using drop() with columns parameter
df_drop_cols = df.drop(columns=['Manager', 'Age_Group'])
print("After dropping Manager and Age_Group columns:")
print(df_drop_cols)

# Method 2: Using del statement
df_del = df.copy()
del df_del['Performance']
print("\nAfter deleting Performance column using del:")
print(df_del)

# Method 3: Using pop() method (returns the column)
df_pop = df.copy()
popped_column = df_pop.pop('Employee_ID')
print(f"\nPopped column (Employee_ID):\n{popped_column}")
print("\nDataFrame after popping Employee_ID:")
print(df_pop)

# Adding rows
print(f"\n--- Adding Rows ---")

# Method 1: Using loc with new index
df.loc['F'] = ['Frank', 45, 'Miami', 80000, 'Finance', 12, 'Excellent', '']
print("After adding Frank's row using loc:")
print(df)

# Method 2: Using append() method

```

```

new_row = pd.Series(['Grace', 29, 'Seattle', 58000, 'HR', 4, 'Good', 'Tom',
                    index=df.columns, name='G')
df = df.append(new_row)
print("\nAfter adding Grace's row using append:")
print(df)

# Method 3: Using concat() method
new_rows = pd.DataFrame({
    'Name': ['Henry', 'Ivy'],
    'Age': [38, 26],
    'City': ['Boston', 'Denver'],
    'Salary': [72000, 62000],
    'Department': ['IT', 'Marketing'],
    'Experience': [9, 1],
    'Performance': ['Excellent', 'Good'],
    'Manager': ['John', 'Lisa'],
    'Salary_Plus_Bonus': [79200, 68200],
    'Age_Group': ['Senior', 'Young']
}, index=['H', 'I'])
df = pd.concat([df, new_rows])
print("\nAfter adding Henry and Ivy using concat:")
print(df)

# Removing rows
print(f"\n--- Removing Rows ---")

# Method 1: Using drop() with index
df_drop_rows = df.drop(['A', 'C'])
print("After dropping rows A and C:")
print(df_drop_rows)

# Method 2: Using drop() with conditions
df_condition_drop = df.drop(df[df['Age'] < 30].index)
print("\nAfter dropping rows where Age < 30:")
print(df_condition_drop)

# Method 3: Using boolean indexing to keep rows
df_keep_rows = df[df['Department'] != 'Marketing']
print("\nAfter keeping only non-Marketing employees:")
print(df_keep_rows)

# Method 4: Using dropna() to remove rows with missing values
df_with_nulls = df.copy()
df_with_nulls.loc['A', 'Salary'] = np.nan
df_with_nulls.loc['C', 'Department'] = np.nan
print("\nDataFrame with some null values:")
print(df_with_nulls[['Name', 'Salary', 'Department']])

df_no_nulls = df_with_nulls.dropna()
print("\nAfter removing rows with any null values:")
print(df_no_nulls[['Name', 'Salary', 'Department']])

# Method 5: Using dropna() with specific columns
df_drop_specific_nulls = df_with_nulls.dropna(subset=['Salary'])
print("\nAfter removing rows with null Salary:")
print(df_drop_specific_nulls[['Name', 'Salary', 'Department']])

# Reindexing
print(f"\n--- Reindexing ---")

```

```

# Reset index
df_reset = df.reset_index()
print("After resetting index:")
print(df_reset.head())

# Set new index
df_new_index = df.set_index('Name')
print("\nAfter setting Name as index:")
print(df_new_index.head())

# Reorder columns
print(f"\n--- Reordering Columns ---")
df_reordered = df[['Name', 'Age', 'Department', 'Salary', 'City', 'Experience']]
print("After reordering columns:")
print(df_reordered.head())

# Using reindex() for columns
df_reindex_cols = df.reindex(columns=['Name', 'Salary', 'Age', 'Department'])
print("\nAfter reindexing columns:")
print(df_reindex_cols.head())

# Using reindex() for rows
df_reindex_rows = df.reindex(['A', 'B', 'C', 'F', 'G', 'H', 'I', 'J'])
print("\nAfter reindexing rows (with some missing):")
print(df_reindex_rows.head())

# Renaming columns and index
print(f"\n--- Renaming ---")

# Rename columns
df_renamed_cols = df.rename(columns={'Salary': 'Annual_Salary', 'City': 'Location'})
print("After renaming columns:")
print(df_renamed_cols.head())

# Rename index
df_renamed_idx = df.rename(index={'A': 'Emp_A', 'B': 'Emp_B'})
print("\nAfter renaming index:")
print(df_renamed_idx.head())

# Rename both
df_renamed_both = df.rename(columns={'Salary': 'Annual_Salary'}, index={'A': 'Emp_A', 'B': 'Emp_B'})
print("\nAfter renaming both columns and index:")
print(df_renamed_both.head())

```

7. Handling Missing Values

```

In [ ]: # Create a DataFrame with various types of missing values
data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'Diana', 'Eve', 'Frank', 'Grace', 'Heidi', 'Ivy'],
    'Age': [25, 30, np.nan, 28, 32, 45, np.nan, 38, 22],
    'City': ['New York', 'Los Angeles', 'Chicago', np.nan, 'Phoenix', 'Miami', 'San Francisco', 'Seattle', 'Boston'],
    'Salary': [50000, 60000, 70000, 55000, np.nan, 80000, 58000, 72000, 65000],
    'Department': ['IT', 'HR', 'IT', 'Finance', 'IT', 'Finance', 'HR', 'IT', 'Finance'],
    'Experience': [2, 5, 8, 3, 6, 12, 4, 9, 7],
    'Performance': ['Good', 'Excellent', np.nan, 'Average', 'Excellent', 'Good', 'Average', 'Good', 'Excellent']
}
df = pd.DataFrame(data, index=['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'I'])

```

```

print("DataFrame with missing values:")
print(df)

# Detecting missing values
print(f"\n--- Detecting Missing Values ---")

# Check for missing values
print("Missing values per column:")
print(df.isnull().sum())

print("\nMissing values as percentage:")
print((df.isnull().sum() / len(df) * 100).round(2))

print("\nAny missing values in each row:")
print(df.isnull().any(axis=1))

print("\nRows with any missing values:")
print(df[df.isnull().any(axis=1)])

# Check for non-missing values
print(f"\n--- Non-Missing Values ---")
print("Non-missing values per column:")
print(df.notnull().sum())

print("\nRows with all values present:")
print(df[df.notnull().all(axis=1)])

# Filling missing values
print(f"\n--- Filling Missing Values ---")

# Fill with constant values
df_fill_const = df.copy()
df_fill_const['Age'] = df_fill_const['Age'].fillna(30) # Fill age with 30
df_fill_const['City'] = df_fill_const['City'].fillna('Unknown') # Fill city with Unknown
print("After filling with constants:")
print(df_fill_const[['Name', 'Age', 'City']])

# Fill with statistical measures
df_fill_stats = df.copy()
df_fill_stats['Age'] = df_fill_stats['Age'].fillna(df_fill_stats['Age'].mean())
df_fill_stats['Salary'] = df_fill_stats['Salary'].fillna(df_fill_stats['Salary'].mean())
print("\nAfter filling with statistical measures:")
print(df_fill_stats[['Name', 'Age', 'Salary']])

# Fill with forward fill (ffill)
df_fill_forward = df.copy()
df_fill_forward['Performance'] = df_fill_forward['Performance'].fillna(method='ffill')
print("\nAfter forward fill for Performance:")
print(df_fill_forward[['Name', 'Performance']])

# Fill with backward fill (bfill)
df_fill_backward = df.copy()
df_fill_backward['Performance'] = df_fill_backward['Performance'].fillna(method='bfill')
print("\nAfter backward fill for Performance:")
print(df_fill_backward[['Name', 'Performance']])

# Fill with interpolate for numeric data
df_interpolate = df.copy()
df_interpolate['Age'] = df_interpolate['Age'].interpolate()
df_interpolate['Salary'] = df_interpolate['Salary'].interpolate()

```

```

print("\nAfter interpolation for numeric columns:")
print(df_interpolate[['Name', 'Age', 'Salary']])

# Fill with mode for categorical data
df_fill_mode = df.copy()
mode_performance = df_fill_mode['Performance'].mode()[0]
df_fill_mode['Performance'] = df_fill_mode['Performance'].fillna(mode_performance)
print(f"\nAfter filling Performance with mode ({mode_performance}):")
print(df_fill_mode[['Name', 'Performance']])

# Using fillna() with different methods for different columns
df_fill_multiple = df.copy()
df_fill_multiple = df_fill_multiple.fillna({
    'Age': df_fill_multiple['Age'].mean(),
    'City': 'Unknown',
    'Salary': df_fill_multiple['Salary'].median(),
    'Performance': 'Good'
})
print("\nAfter filling with different methods for different columns:")
print(df_fill_multiple)

# Dropping missing values
print(f"\n--- Dropping Missing Values ---")

# Drop rows with any missing values
df_drop_any = df.dropna()
print("After dropping rows with any missing values:")
print(df_drop_any)

# Drop rows with all missing values
df_drop_all = df.copy()
df_drop_all.loc['I'] = [np.nan, np.nan, np.nan, np.nan, np.nan, np.nan, np.nan]
print("\nDataFrame with a row of all NaN:")
print(df_drop_all)

df_drop_all_nan = df_drop_all.dropna(how='all')
print("\nAfter dropping rows with all missing values:")
print(df_drop_all_nan)

# Drop columns with any missing values
df_drop_cols = df.dropna(axis=1)
print("\nAfter dropping columns with any missing values:")
print(df_drop_cols)

# Drop rows with missing values in specific columns
df_drop_specific = df.dropna(subset=['Age', 'Salary'])
print("\nAfter dropping rows with missing Age or Salary:")
print(df_drop_specific)

# Drop rows with missing values in all specified columns
df_drop_all_specific = df.dropna(subset=['Age', 'Salary'], how='all')
print("\nAfter dropping rows with missing Age AND Salary:")
print(df_drop_all_specific)

# Using thresh parameter
df_drop_thresh = df.dropna(thresh=6) # Keep rows with at least 6 non-null
print("\nAfter keeping rows with at least 6 non-null values:")
print(df_drop_thresh)

# Working with missing values in specific contexts

```

```

print(f"\n--- Working with Missing Values in Context ---")

# Replace specific values with NaN
df_replace = df.copy()
df_replace['Age'] = df_replace['Age'].replace(45, np.nan) # Replace 45 w
print("\nAfter replacing 45 with NaN in Age:")
print(df_replace[['Name', 'Age']])

# Replace multiple values
df_replace_multiple = df.copy()
df_replace_multiple['Performance'] = df_replace_multiple['Performance'].r
print("\nAfter replacing 'Good' and 'Excellent' with NaN in Performance:")
print(df_replace_multiple[['Name', 'Performance']])

# Check for missing values in specific data types
print(f"\n--- Missing Values by Data Type ---")
print("Missing values in numeric columns:")
numeric_cols = df.select_dtypes(include=[np.number]).columns
print(df[numeric_cols].isnull().sum())

print("\nMissing values in object columns:")
object_cols = df.select_dtypes(include=['object']).columns
print(df[object_cols].isnull().sum())

# Advanced missing value handling
print(f"\n--- Advanced Missing Value Handling ---")

# Create a more complex example
df_complex = pd.DataFrame({
    'A': [1, 2, np.nan, 4, 5],
    'B': [np.nan, 2, 3, np.nan, 5],
    'C': [1, np.nan, 3, 4, np.nan],
    'D': [1, 2, 3, 4, 5]
})

print("Complex DataFrame:")
print(df_complex)

# Fill missing values with the mean of each column
df_fill_mean = df_complex.fillna(df_complex.mean())
print("\nAfter filling with column means:")
print(df_fill_mean)

# Fill missing values with the median of each column
df_fill_median = df_complex.fillna(df_complex.median())
print("\nAfter filling with column medians:")
print(df_fill_median)

# Fill missing values with the mode of each column
df_fill_mode = df_complex.fillna(df_complex.mode().iloc[0])
print("\nAfter filling with column modes:")
print(df_fill_mode)

# Fill missing values with the previous value (forward fill)
df_fill_ffill = df_complex.fillna(method='ffill')
print("\nAfter forward fill:")
print(df_fill_ffill)

# Fill missing values with the next value (backward fill)
df_fill_bfill = df_complex.fillna(method='bfill')

```

```

print("\nAfter backward fill:")
print(df_fill_bfill)

# Fill missing values with interpolated values
df_fill_interpolate = df_complex.interpolate()
print("\nAfter interpolation:")
print(df_fill_interpolate)

```

8. Data Type Conversion

```

In [ ]: # Create a DataFrame with mixed data types for conversion examples
data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'Diana', 'Eve'],
    'Age': ['25', '30', '35', '28', '32'], # String numbers
    'Salary': [50000.0, 60000.0, 70000.0, 55000.0, 65000.0], # Float
    'Is_Manager': [1, 0, 1, 0, 1], # Integer boolean
    'Department': ['IT', 'HR', 'IT', 'Finance', 'IT'],
    'Join_Date': ['2020-01-15', '2019-03-22', '2018-07-10', '2021-02-28',
    'Rating': ['4.5', '3.8', '4.2', '4.0', '4.7'] # String numbers
}
df = pd.DataFrame(data)

print("Original DataFrame with mixed data types:")
print(df)
print(f"\nData types:\n{df.dtypes}")

# Converting to different data types
print(f"\n--- Converting Data Types ---")

# Convert string numbers to numeric
df['Age'] = pd.to_numeric(df['Age'])
print("After converting Age to numeric:")
print(f"Age data type: {df['Age'].dtype}")
print(df['Age'])

# Convert with error handling
df['Rating'] = pd.to_numeric(df['Rating'], errors='coerce')
print(f"\nAfter converting Rating to numeric (with error handling):")
print(f"Rating data type: {df['Rating'].dtype}")
print(df['Rating'])

# Convert to integer
df['Age'] = df['Age'].astype(int)
print(f"\nAfter converting Age to integer:")
print(f"Age data type: {df['Age'].dtype}")
print(df['Age'])

# Convert to float
df['Rating'] = df['Rating'].astype(float)
print(f"\nAfter converting Rating to float:")
print(f"Rating data type: {df['Rating'].dtype}")
print(df['Rating'])

# Convert to string
df['Salary'] = df['Salary'].astype(str)
print(f"\nAfter converting Salary to string:")
print(f"Salary data type: {df['Salary'].dtype}")
print(df['Salary'])

```



```

# Convert to boolean
df['Is_Manager'] = df['Is_Manager'].astype(bool)
print(f"\nAfter converting Is_Manager to boolean:")
print(f"Is_Manager data type: {df['Is_Manager'].dtype}")
print(df['Is_Manager'])

# Convert to datetime
df['Join_Date'] = pd.to_datetime(df['Join_Date'])
print(f"\nAfter converting Join_Date to datetime:")
print(f"Join_Date data type: {df['Join_Date'].dtype}")
print(df['Join_Date'])

# Convert to category
df['Department'] = df['Department'].astype('category')
print(f"\nAfter converting Department to category:")
print(f"Department data type: {df['Department'].dtype}")
print(df['Department'])
print(f"Categories: {df['Department'].cat.categories}")

# Using astype() for multiple columns
print(f"\n--- Converting Multiple Columns ---")
df_multi = df.copy()
df_multi[['Age', 'Salary']] = df_multi[['Age', 'Salary']].astype(float)
print("After converting Age and Salary to float:")
print(df_multi[['Age', 'Salary']].dtypes)

# Using to_numeric() with different parameters
print(f"\n--- Using to_numeric() Parameters ---")

# Create data with some invalid values
data_invalid = {
    'A': ['1', '2', 'invalid', '4', '5'],
    'B': ['1.5', '2.7', '3.2', 'not_a_number', '5.1'],
    'C': ['10', '20', '30', '40', '50']
}
df_invalid = pd.DataFrame(data_invalid)

print("DataFrame with invalid numeric values:")
print(df_invalid)

# Convert with errors='coerce' (invalid becomes NaN)
df_coerce = df_invalid.copy()
df_coerce['A'] = pd.to_numeric(df_coerce['A'], errors='coerce')
df_coerce['B'] = pd.to_numeric(df_coerce['B'], errors='coerce')
print("\nAfter converting with errors='coerce':")
print(df_coerce)

# Convert with errors='ignore' (keeps as string)
df_ignore = df_invalid.copy()
df_ignore['A'] = pd.to_numeric(df_ignore['A'], errors='ignore')
print("\nAfter converting with errors='ignore':")
print(df_ignore['A'])

# Convert with downcast parameter
df_downcast = df_invalid.copy()
df_downcast['C'] = pd.to_numeric(df_downcast['C'], downcast='integer')
print(f"\nAfter converting C with downcast='integer':")
print(f"C data type: {df_downcast['C'].dtype}")

```

```

# Using infer_objects() to automatically infer types
print(f"\n--- Using infer_objects() ---")
df_infer = df.copy()
df_infer['Age'] = df_infer['Age'].astype(str) # Convert back to string
df_infer['Salary'] = df_infer['Salary'].astype(str) # Convert back to string
print("DataFrame with string numbers:")
print(df_infer[['Age', 'Salary']].dtypes)

df_infer = df_infer.infer_objects()
print("\nAfter using infer_objects():")
print(df_infer[['Age', 'Salary']].dtypes)

# Converting datetime with different formats
print(f"\n--- DateTime Conversion ---")
date_data = {
    'Date1': ['2023-01-15', '2023-02-20', '2023-03-25'],
    'Date2': ['15/01/2023', '20/02/2023', '25/03/2023'],
    'Date3': ['Jan 15, 2023', 'Feb 20, 2023', 'Mar 25, 2023'],
    'DateTime': ['2023-01-15 10:30:00', '2023-02-20 14:45:00', '2023-03-25 09:15:00']
}
df_dates = pd.DataFrame(date_data)

print("DataFrame with different date formats:")
print(df_dates)

# Convert with default format
df_dates['Date1'] = pd.to_datetime(df_dates['Date1'])
print(f"\nDate1 after conversion: {df_dates['Date1'].dtype}")

# Convert with specific format
df_dates['Date2'] = pd.to_datetime(df_dates['Date2'], format='%d/%m/%Y')
print(f"Date2 after conversion: {df_dates['Date2'].dtype}")

# Convert with infer_datetime_format
df_dates['Date3'] = pd.to_datetime(df_dates['Date3'], infer_datetime_format=True)
print(f"Date3 after conversion: {df_dates['Date3'].dtype}")

# Convert datetime string
df_dates['DateTime'] = pd.to_datetime(df_dates['DateTime'])
print(f"DateTime after conversion: {df_dates['DateTime'].dtype}")

print("\nConverted dates:")
print(df_dates)

# Converting to categorical with custom categories
print(f"\n--- Categorical Conversion ---")
df_cat = df.copy()
df_cat['Department'] = pd.Categorical(df_cat['Department'],
                                     categories=['HR', 'IT', 'Finance', 'Marketing'],
                                     ordered=True)

print("Department as ordered categorical:")
print(df_cat['Department'])
print(f"Categories: {df_cat['Department'].cat.categories}")
print(f"Is ordered: {df_cat['Department'].cat.ordered}")

# Converting boolean with different representations
print(f"\n--- Boolean Conversion ---")
bool_data = {
    'Yes_No': ['Yes', 'No', 'Yes', 'No', 'Yes'],
    'True_False': ['True', 'False', 'True', 'False', 'True'],
}
df_bool = pd.DataFrame(bool_data)
df_bool['Yes_No'] = df_bool['Yes_No'].astype(bool)
df_bool['True_False'] = df_bool['True_False'].astype(bool)
print(df_bool)

```

```

    'Numbers': [1, 0, 1, 0, 1],
    'Strings': ['true', 'false', 'true', 'false', 'true']
}
df_bool = pd.DataFrame(bool_data)

print("DataFrame with different boolean representations:")
print(df_bool)

# Convert different boolean representations
df_bool['Yes_No'] = df_bool['Yes_No'].map({'Yes': True, 'No': False})
df_bool['True_False'] = df_bool['True_False'].map({'True': True, 'False': False})
df_bool['Numbers'] = df_bool['Numbers'].astype(bool)
df_bool['Strings'] = df_bool['Strings'].map({'true': True, 'false': False})

print("\nAfter converting to boolean:")
print(df_bool)
print(f"\nData types:\n{df_bool.dtypes}")

# Using convert_dtypes() for optimal data types
print(f"\n--- Using convert_dtypes() ---")
df_convert = df.copy()
df_convert['Age'] = df_convert['Age'].astype(str)
df_convert['Salary'] = df_convert['Salary'].astype(str)
df_convert['Is_Manager'] = df_convert['Is_Manager'].astype(str)

print("DataFrame with suboptimal data types:")
print(df_convert.dtypes)

df_convert = df_convert.convert_dtypes()
print("\nAfter using convert_dtypes():")
print(df_convert.dtypes)

```

9. Sorting and Ranking

```

In [ ]: # Create a comprehensive DataFrame for sorting and ranking examples
data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'Diana', 'Eve', 'Frank', 'Grace', 'Helen'],
    'Age': [25, 30, 35, 28, 32, 45, 29, 38],
    'City': ['New York', 'Los Angeles', 'Chicago', 'Houston', 'Phoenix', 'San Francisco', 'Seattle', 'Miami'],
    'Salary': [50000, 60000, 70000, 55000, 65000, 80000, 58000, 72000],
    'Department': ['IT', 'HR', 'IT', 'Finance', 'IT', 'Finance', 'HR', 'IT'],
    'Experience': [2, 5, 8, 3, 6, 12, 4, 9],
    'Performance': ['Good', 'Excellent', 'Good', 'Average', 'Excellent', 'Good', 'Average', 'Good']
}
df = pd.DataFrame(data, index=['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H'])

print("Original DataFrame:")
print(df)

# Basic sorting
print(f"\n--- Basic Sorting ---")

# Sort by a single column (ascending)
df_sorted_age = df.sort_values('Age')
print("Sorted by Age (ascending):")
print(df_sorted_age)

# Sort by a single column (descending)

```

```

df_sorted_salary = df.sort_values('Salary', ascending=False)
print("\nSorted by Salary (descending):")
print(df_sorted_salary)

# Sort by multiple columns
df_sorted_multi = df.sort_values(['Department', 'Salary'], ascending=True)
print("\nSorted by Department (ascending) then Salary (descending):")
print(df_sorted_multi)

# Sort by index
df_sorted_index = df.sort_index()
print("\nSorted by index:")
print(df_sorted_index)

# Sort by index in descending order
df_sorted_index_desc = df.sort_index(ascending=False)
print("\nSorted by index (descending):")
print(df_sorted_index_desc)

# Sorting with missing values
print(f"\n--- Sorting with Missing Values ---")

# Create DataFrame with missing values
df_with_nulls = df.copy()
df_with_nulls.loc['A', 'Salary'] = np.nan
df_with_nulls.loc['C', 'Age'] = np.nan

print("DataFrame with missing values:")
print(df_with_nulls[['Name', 'Age', 'Salary']])

# Sort with na_position='first' (missing values first)
df_nulls_first = df_with_nulls.sort_values('Salary', na_position='first')
print("\nSorted by Salary with missing values first:")
print(df_nulls_first[['Name', 'Age', 'Salary']])

# Sort with na_position='last' (missing values last)
df_nulls_last = df_with_nulls.sort_values('Age', na_position='last')
print("\nSorted by Age with missing values last:")
print(df_nulls_last[['Name', 'Age', 'Salary']])

# Custom sorting
print(f"\n--- Custom Sorting ---")

# Sort by custom order for categorical data
performance_order = ['Excellent', 'Good', 'Average']
df_custom = df.copy()
df_custom['Performance'] = pd.Categorical(df_custom['Performance'], categories=performance_order)
df_custom_sorted = df_custom.sort_values('Performance')
print("Sorted by Performance (custom order):")
print(df_custom_sorted[['Name', 'Performance']])

# Sort by string length
df['Name_Length'] = df['Name'].str.len()
df_sorted_length = df.sort_values('Name_Length')
print("\nSorted by Name length:")
print(df_sorted_length[['Name', 'Name_Length']])

# Ranking
print(f"\n--- Ranking ---")

```

```

# Basic ranking
df['Salary_Rank'] = df['Salary'].rank()
print("DataFrame with Salary ranking:")
print(df[['Name', 'Salary', 'Salary_Rank']])

# Ranking with different methods
df['Salary_Rank_Avg'] = df['Salary'].rank(method='average')
df['Salary_Rank_Min'] = df['Salary'].rank(method='min')
df['Salary_Rank_Max'] = df['Salary'].rank(method='max')
df['Salary_Rank_Dense'] = df['Salary'].rank(method='dense')

print("\nDifferent ranking methods for Salary:")
print(df[['Name', 'Salary', 'Salary_Rank_Avg', 'Salary_Rank_Min', 'Salary_Rank_Max', 'Salary_Rank_Dense']])

# Ranking in descending order
df['Salary_Rank_Desc'] = df['Salary'].rank(ascending=False)
print("\nSalary ranking (descending):")
print(df[['Name', 'Salary', 'Salary_Rank_Desc']])

# Ranking with ties
print(f"\n--- Ranking with Ties ---")

# Create data with ties
data_ties = {
    'Name': ['Alice', 'Bob', 'Charlie', 'Diana', 'Eve'],
    'Score': [85, 90, 85, 90, 88]
}
df_ties = pd.DataFrame(data_ties)

print("DataFrame with ties:")
print(df_ties)

# Different methods for handling ties
df_ties['Rank_Average'] = df_ties['Score'].rank(method='average')
df_ties['Rank_Min'] = df_ties['Score'].rank(method='min')
df_ties['Rank_Max'] = df_ties['Score'].rank(method='max')
df_ties['Rank_Dense'] = df_ties['Score'].rank(method='dense')

print("\nDifferent tie-breaking methods:")
print(df_ties)

# Percentile ranking
print(f"\n--- Percentile Ranking ---")
df['Salary_Percentile'] = df['Salary'].rank(pct=True)
print("Salary percentile ranking:")
print(df[['Name', 'Salary', 'Salary_Percentile']])

# Ranking by groups
print(f"\n--- Ranking by Groups ---")
df['Salary_Rank_by_Dept'] = df.groupby('Department')['Salary'].rank(ascending=True)
print("Salary ranking within each department:")
print(df[['Name', 'Department', 'Salary', 'Salary_Rank_by_Dept']])

# Advanced sorting examples
print(f"\n--- Advanced Sorting Examples ---")

# Sort by multiple criteria with different orders
df_advanced = df.sort_values(['Department', 'Experience', 'Salary'],
                             ascending=[True, False, True])
print("Sorted by Department (asc), Experience (desc), Salary (asc):")

```

```

print(df_advanced[['Name', 'Department', 'Experience', 'Salary']])

# Sort by calculated column
df['Salary_Per_Year'] = df['Salary'] / df['Experience']
df_sorted_ratio = df.sort_values('Salary_Per_Year', ascending=False)
print("\nSorted by Salary per year of experience:")
print(df_sorted_ratio[['Name', 'Salary', 'Experience', 'Salary_Per_Year']])

# Sort by string operations
df['City_Length'] = df['City'].str.len()
df_sorted_city = df.sort_values('City_Length')
print("\nSorted by city name length:")
print(df_sorted_city[['Name', 'City', 'City_Length']])

# Sort by multiple string criteria
df_sorted_strings = df.sort_values(['Department', 'Name'])
print("\nSorted by Department then Name:")
print(df_sorted_strings[['Name', 'Department']])

# Using nlargest and nsmallest
print(f"\n--- Using nlargest and nsmallest ---")

# Get top 3 highest salaries
top_3_salaries = df.nlargest(3, 'Salary')
print("Top 3 highest salaries:")
print(top_3_salaries[['Name', 'Salary']])

# Get bottom 3 lowest salaries
bottom_3_salaries = df.nsmallest(3, 'Salary')
print("\nBottom 3 lowest salaries:")
print(bottom_3_salaries[['Name', 'Salary']])

# Get top 2 from each department
top_by_dept = df.groupby('Department').apply(lambda x: x.nlargest(2, 'Salary'))
print("\nTop 2 salaries from each department:")
print(top_by_dept[['Name', 'Department', 'Salary']])

# Sorting with key functions
print(f"\n--- Sorting with Key Functions ---")

# Sort by absolute difference from mean salary
mean_salary = df['Salary'].mean()
df['Salary_Diff_From_Mean'] = abs(df['Salary'] - mean_salary)
df_sorted_diff = df.sort_values('Salary_Diff_From_Mean')
print("Sorted by absolute difference from mean salary:")
print(df_sorted_diff[['Name', 'Salary', 'Salary_Diff_From_Mean']])

# Sort by custom function (e.g., by last letter of name)
df['Name_Last_Letter'] = df['Name'].str[-1]
df_sorted_last_letter = df.sort_values('Name_Last_Letter')
print("\nSorted by last letter of name:")
print(df_sorted_last_letter[['Name', 'Name_Last_Letter']])

```

10. Basic Aggregation Functions

```

In [ ]: # Create a comprehensive DataFrame for aggregation examples
data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'Diana', 'Eve', 'Frank', 'Grace',

```

```

    'Age': [25, 30, 35, 28, 32, 45, 29, 38, 26, 42],
    'City': ['New York', 'Los Angeles', 'Chicago', 'Houston', 'Phoenix',
    'Salary': [50000, 60000, 70000, 55000, 65000, 80000, 58000, 72000, 62
    'Department': ['IT', 'HR', 'IT', 'Finance', 'IT', 'Finance', 'HR', 'I
    'Experience': [2, 5, 8, 3, 6, 12, 4, 9, 1, 10],
    'Performance': ['Good', 'Excellent', 'Good', 'Average', 'Excellent',
    'Bonus': [5000, 8000, 7000, 3000, 9000, 12000, 4000, 10000, 2000, 750
}
df = pd.DataFrame(data, index=['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'I

print("Original DataFrame:")
print(df)

# Basic aggregation functions
print(f"\n--- Basic Aggregation Functions ---")

# Sum
print("Sum of numeric columns:")
print(df[['Age', 'Salary', 'Experience', 'Bonus']].sum())

# Mean
print("\nMean of numeric columns:")
print(df[['Age', 'Salary', 'Experience', 'Bonus']].mean())

# Median
print("\nMedian of numeric columns:")
print(df[['Age', 'Salary', 'Experience', 'Bonus']].median())

# Mode
print("\nMode of categorical columns:")
print(df[['Department', 'Performance']].mode())

# Standard deviation
print("\nStandard deviation of numeric columns:")
print(df[['Age', 'Salary', 'Experience', 'Bonus']].std())

# Variance
print("\nVariance of numeric columns:")
print(df[['Age', 'Salary', 'Experience', 'Bonus']].var())

# Min and Max
print("\nMinimum values:")
print(df[['Age', 'Salary', 'Experience', 'Bonus']].min())

print("\nMaximum values:")
print(df[['Age', 'Salary', 'Experience', 'Bonus']].max())

# Count
print("\nCount of non-null values:")
print(df.count())

# Count unique values
print("\nCount of unique values:")
print(df.nunique())

# Describe method
print(f"\n--- Describe Method ---")
print("Statistical summary of numeric columns:")
print(df.describe())

```

```

print("\nStatistical summary including all columns:")
print(df.describe(include='all'))

# Aggregation with groupby
print(f"\n--- Aggregation with GroupBy ---")

# Group by Department
dept_stats = df.groupby('Department').agg({
    'Salary': ['mean', 'median', 'std', 'count'],
    'Age': ['mean', 'min', 'max'],
    'Experience': ['mean', 'sum']
})
print("Statistics by Department:")
print(dept_stats)

# Flatten column names
dept_stats.columns = ['_'.join(col).strip() for col in dept_stats.columns]
print("\nFlattened column names:")
print(dept_stats)

# Multiple aggregation functions on same column
print(f"\n--- Multiple Aggregations on Same Column ---")
salary_stats = df.groupby('Department')['Salary'].agg(['count', 'mean', '
print("Salary statistics by Department:")
print(salary_stats)

# Custom aggregation functions
print(f"\n--- Custom Aggregation Functions ---")

# Define custom functions
def range_func(x):
    return x.max() - x.min()

def iqr_func(x):
    return x.quantile(0.75) - x.quantile(0.25)

# Apply custom functions
custom_stats = df.groupby('Department')['Salary'].agg(['mean', 'std', ran
custom_stats.columns = ['Mean', 'Std', 'Range', 'IQR']
print("Custom salary statistics by Department:")
print(custom_stats)

# Aggregation with multiple columns
print(f"\n--- Aggregation with Multiple Columns ---")
multi_col_stats = df.groupby('Department')[['Salary', 'Age', 'Experience'
print("Multiple column statistics by Department:")
print(multi_col_stats)

# Using apply with custom functions
print(f"\n--- Using Apply with Custom Functions ---")

# Custom function for salary analysis
def salary_analysis(group):
    return pd.Series({
        'avg_salary': group['Salary'].mean(),
        'max_salary': group['Salary'].max(),
        'min_salary': group['Salary'].min(),
        'salary_range': group['Salary'].max() - group['Salary'].min(),
        'employee_count': len(group)
    })

```



```

salary_analysis_result = df.groupby('Department').apply(salary_analysis)
print("Custom salary analysis by Department:")
print(salary_analysis_result)

# Aggregation with filtering
print(f"\n--- Aggregation with Filtering ---")

# Filter data before aggregation
high_performers = df[df['Performance'].isin(['Excellent', 'Good'])]
high_perf_stats = high_performers.groupby('Department')['Salary'].agg(['m
print("Salary statistics for high performers by Department:")
print(high_perf_stats)

# Conditional aggregation
print(f"\n--- Conditional Aggregation ---")

# Count by condition
dept_high_salary = df.groupby('Department').apply(
    lambda x: (x['Salary'] > 60000).sum()
)
print("Count of employees with salary > 60000 by Department:")
print(dept_high_salary)

# Percentage by condition
dept_high_salary_pct = df.groupby('Department').apply(
    lambda x: (x['Salary'] > 60000).mean() * 100
)
print("\nPercentage of employees with salary > 60000 by Department:")
print(dept_high_salary_pct)

# Aggregation with multiple groupby columns
print(f"\n--- Multiple GroupBy Columns ---")

# Group by Department and Performance
dept_perf_stats = df.groupby(['Department', 'Performance'])['Salary'].agg
print("Salary statistics by Department and Performance:")
print(dept_perf_stats)

# Pivot table style aggregation
print(f"\n--- Pivot Table Style Aggregation ---")
pivot_stats = df.pivot_table(
    values='Salary',
    index='Department',
    columns='Performance',
    aggfunc=['mean', 'count'],
    fill_value=0
)
print("Pivot table of Salary by Department and Performance:")
print(pivot_stats)

# Cross-tabulation
print(f"\n--- Cross-tabulation ---")
crosstab = pd.crosstab(df['Department'], df['Performance'])
print("Cross-tabulation of Department and Performance:")
print(crosstab)

# Aggregation with time series (if applicable)
print(f"\n--- Time Series Aggregation ---")

```

```

# Create a time series example
dates = pd.date_range('2023-01-01', periods=10, freq='D')
df_time = df.copy()
df_time['Date'] = dates
df_time['Daily_Sales'] = np.random.randint(1000, 5000, 10)

print("Time series DataFrame:")
print(df_time[['Name', 'Date', 'Daily_Sales']].head())

# Resample by week
weekly_sales = df_time.set_index('Date')['Daily_Sales'].resample('W').sum
print("\nWeekly sales totals:")
print(weekly_sales)

# Rolling window aggregation
print(f"\n--- Rolling Window Aggregation ---")
df_time_sorted = df_time.sort_values('Date')
df_time_sorted['Rolling_Avg'] = df_time_sorted['Daily_Sales'].rolling(win
print("Daily sales with 3-day rolling average:")
print(df_time_sorted[['Date', 'Daily_Sales', 'Rolling_Avg']].head())

# Expanding window aggregation
print(f"\n--- Expanding Window Aggregation ---")
df_time_sorted['Expanding_Sum'] = df_time_sorted['Daily_Sales'].expanding
print("Daily sales with expanding sum:")
print(df_time_sorted[['Date', 'Daily_Sales', 'Expanding_Sum']].head())

# Aggregation with missing values
print(f"\n--- Aggregation with Missing Values ---")

# Create DataFrame with missing values
df_missing = df.copy()
df_missing.loc['A', 'Salary'] = np.nan
df_missing.loc['C', 'Age'] = np.nan

print("DataFrame with missing values:")
print(df_missing[['Name', 'Age', 'Salary']].head())

# Aggregation with skipna=True (default)
print("\nMean with skipna=True (default):")
print(df_missing[['Age', 'Salary']].mean())

# Aggregation with skipna=False
print("\nMean with skipna=False:")
print(df_missing[['Age', 'Salary']].mean(skipna=False))

# Count non-null values
print("\nCount of non-null values:")
print(df_missing[['Age', 'Salary']].count())

# Advanced aggregation examples
print(f"\n--- Advanced Aggregation Examples ---")

# Weighted average
df['Weight'] = np.random.rand(10)
weighted_avg = (df['Salary'] * df['Weight']).sum() / df['Weight'].sum()
print(f"Weighted average salary: {weighted_avg:.2f}")

# Percentile aggregation
percentiles = df.groupby('Department')['Salary'].quantile([0.25, 0.5, 0.7

```

```

print("\nSalary percentiles by Department:")
print(percentiles)

# Aggregation with custom grouping
print(f"\n--- Custom Grouping ---")

# Create age groups
df['Age_Group'] = pd.cut(df['Age'], bins=[0, 30, 40, 100], labels=['Young', 'Middle', 'Old'])
age_group_stats = df.groupby('Age_Group')['Salary'].agg(['mean', 'count'])
print("Salary statistics by Age Group:")
print(age_group_stats)

```

11. DataFrame Operations and Arithmetic

```

In [ ]: # Create DataFrames for arithmetic operations
data1 = {
    'A': [1, 2, 3, 4, 5],
    'B': [10, 20, 30, 40, 50],
    'C': [100, 200, 300, 400, 500]
}
df1 = pd.DataFrame(data1, index=['X', 'Y', 'Z', 'W', 'V'])

data2 = {
    'A': [2, 4, 6, 8, 10],
    'B': [5, 10, 15, 20, 25],
    'C': [50, 100, 150, 200, 250]
}
df2 = pd.DataFrame(data2, index=['X', 'Y', 'Z', 'W', 'V'])

print("DataFrame 1:")
print(df1)
print("\nDataFrame 2:")
print(df2)

# Basic arithmetic operations
print(f"\n--- Basic Arithmetic Operations ---")

# Addition
print("Addition (df1 + df2):")
print(df1 + df2)

# Subtraction
print("\nSubtraction (df1 - df2):")
print(df1 - df2)

# Multiplication
print("\nMultiplication (df1 * df2):")
print(df1 * df2)

# Division
print("\nDivision (df1 / df2):")
print(df1 / df2)

# Floor division
print("\nFloor division (df1 // df2):")
print(df1 // df2)

# Modulo

```

```

print("\nModulo (df1 % df2):")
print(df2 % df1)

# Power
print("\nPower (df1 ** 2):")
print(df1 ** 2)

# Arithmetic with scalars
print(f"\n--- Arithmetic with Scalars ---")

# Add scalar to all elements
print("Add 10 to all elements:")
print(df1 + 10)

# Multiply by scalar
print("\nMultiply by 2:")
print(df1 * 2)

# Divide by scalar
print("\nDivide by 5:")
print(df1 / 5)

# Arithmetic with Series
print(f"\n--- Arithmetic with Series ---")

# Create a Series
series1 = pd.Series([1, 2, 3, 4, 5], index=['X', 'Y', 'Z', 'W', 'V'])
print("Series:")
print(series1)

# Add Series to DataFrame
print("\nDataFrame + Series:")
print(df1 + series1)

# Multiply DataFrame by Series
print("\nDataFrame * Series:")
print(df1 * series1)

# Broadcasting with different indices
print(f"\n--- Broadcasting with Different Indices ---")

# Series with different index
series2 = pd.Series([10, 20], index=['X', 'Y'])
print("Series with different index:")
print(series2)

# Broadcasting
print("\nDataFrame + Series (broadcasting):")
print(df1 + series2)

# Arithmetic with mismatched DataFrames
print(f"\n--- Arithmetic with Mismatched DataFrames ---")

# Create DataFrame with different columns
df3 = pd.DataFrame({
    'A': [1, 2, 3],
    'D': [100, 200, 300]
}, index=['X', 'Y', 'Z'])

print("DataFrame 3:")

```

```

print(df3)

# Addition with mismatched columns
print("\nAddition with mismatched columns:")
print(df1 + df3)

# Arithmetic with different shapes
print(f"\n--- Arithmetic with Different Shapes ---")

# Create smaller DataFrame
df4 = pd.DataFrame({
    'A': [1, 2],
    'B': [10, 20]
}, index=['X', 'Y'])

print("Smaller DataFrame:")
print(df4)

# Addition with different shapes
print("\nAddition with different shapes:")
print(df1 + df4)

# Using add(), sub(), mul(), div() methods
print(f"\n--- Using Arithmetic Methods ---")

# Add method
print("Using add() method:")
print(df1.add(df2))

# Subtract method
print("\nUsing sub() method:")
print(df1.sub(df2))

# Multiply method
print("\nUsing mul() method:")
print(df1.mul(df2))

# Divide method
print("\nUsing div() method:")
print(df1.div(df2))

# Arithmetic methods with fill_value
print(f"\n--- Arithmetic Methods with fill_value ---")

# Create DataFrames with missing values
df5 = pd.DataFrame({
    'A': [1, 2, np.nan, 4],
    'B': [10, np.nan, 30, 40]
}, index=['X', 'Y', 'Z', 'W'])

df6 = pd.DataFrame({
    'A': [2, 4, 6, 8],
    'B': [5, 10, 15, 20]
}, index=['X', 'Y', 'Z', 'W'])

print("DataFrame with missing values:")
print(df5)
print("\nDataFrame without missing values:")
print(df6)

```

```

# Addition with fill_value
print("\nAddition with fill_value=0:")
print(df5.add(df6, fill_value=0))

# Comparison operations
print(f"\n--- Comparison Operations ---")

# Greater than
print("df1 > df2:")
print(df1 > df2)

# Less than or equal
print("\ndf1 <= df2:")
print(df1 <= df2)

# Equal to
print("\ndf1 == df2:")
print(df1 == df2)

# Not equal to
print("\ndf1 != df2:")
print(df1 != df2)

# Comparison with scalars
print(f"\n--- Comparison with Scalars ---")

# Greater than scalar
print("df1 > 3:")
print(df1 > 3)

# Between values
print("\nValues between 2 and 4:")
print((df1 > 2) & (df1 < 4))

# Logical operations
print(f"\n--- Logical Operations ---")

# AND operation
print("df1 > 2 AND df1 < 4:")
print((df1 > 2) & (df1 < 4))

# OR operation
print("\ndf1 < 2 OR df1 > 4:")
print((df1 < 2) | (df1 > 4))

# NOT operation
print("\nNOT (df1 > 3):")
print(~(df1 > 3))

# String operations
print(f"\n--- String Operations ---")

# Create DataFrame with string data
str_data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'Diana'],
    'City': ['New York', 'Los Angeles', 'Chicago', 'Houston'],
    'Department': ['IT', 'HR', 'IT', 'Finance']
}
df_str = pd.DataFrame(str_data)

```

```

print("String DataFrame:")
print(df_str)

# String concatenation
df_str['Full_Info'] = df_str['Name'] + ' from ' + df_str['City']
print("\nAfter string concatenation:")
print(df_str['Full_Info'])

# String methods
print("\nString methods:")
print("Names in uppercase:", df_str['Name'].str.upper().tolist())
print("Cities in lowercase:", df_str['City'].str.lower().tolist())
print("Name lengths:", df_str['Name'].str.len().tolist())

# Mathematical functions
print(f"\n--- Mathematical Functions ---")

# Absolute value
print("Absolute value of df1 - df2:")
print(np.abs(df1 - df2))

# Square root
print("\nSquare root of df1:")
print(np.sqrt(df1))

# Logarithm
print("\nNatural logarithm of df1:")
print(np.log(df1))

# Trigonometric functions
print("\nSine of df1 (in radians):")
print(np.sin(df1))

# Statistical functions
print(f"\n--- Statistical Functions ---")

# Mean across rows
print("Mean across rows:")
print(df1.mean(axis=1))

# Sum across columns
print("\nSum across columns:")
print(df1.sum(axis=0))

# Cumulative operations
print(f"\n--- Cumulative Operations ---")

# Cumulative sum
print("Cumulative sum:")
print(df1.cumsum())

# Cumulative product
print("\nCumulative product:")
print(df1.cumprod())

# Cumulative maximum
print("\nCumulative maximum:")
print(df1.cummax())

# Cumulative minimum

```

```

print("\nCumulative minimum:")
print(df1.cummin())

# Window functions
print(f"\n--- Window Functions ---")

# Rolling window
print("3-period rolling mean:")
print(df1.rolling(window=3).mean())

# Expanding window
print("\nExpanding mean:")
print(df1.expanding().mean())

# Exponential weighted functions
print("\nExponential weighted mean (alpha=0.5):")
print(df1.ewm(alpha=0.5).mean())

# Custom operations
print(f"\n--- Custom Operations ---")

# Apply custom function
def custom_func(x):
    return x * 2 + 1

print("Custom function (x * 2 + 1):")
print(df1.apply(custom_func))

# Apply function to each column
print("\nApply function to each column:")
print(df1.apply(lambda x: x.max() - x.min()))

# Apply function to each row
print("\nApply function to each row:")
print(df1.apply(lambda x: x.max() - x.min(), axis=1))

# Element-wise operations with applymap
print("\nElement-wise operations with applymap:")
print(df1.applymap(lambda x: x ** 2))

# Vectorized operations
print(f"\n--- Vectorized Operations ---")

# Create a more complex example
df_complex = pd.DataFrame({
    'Sales': [1000, 1500, 2000, 1200, 1800],
    'Cost': [600, 900, 1200, 720, 1080],
    'Quantity': [10, 15, 20, 12, 18]
}, index=['Jan', 'Feb', 'Mar', 'Apr', 'May'])

print("Complex DataFrame:")
print(df_complex)

# Calculate profit margin
df_complex['Profit'] = df_complex['Sales'] - df_complex['Cost']
df_complex['Profit_Margin'] = (df_complex['Profit'] / df_complex['Sales'])
df_complex['Unit_Price'] = df_complex['Sales'] / df_complex['Quantity']

print("\nAfter calculations:")
print(df_complex)

```



```

# Conditional operations
print(f"\n--- Conditional Operations ---")

# Where condition
print("Where Sales > 1500, otherwise 0:")
print(df_complex['Sales'].where(df_complex['Sales'] > 1500, 0))

# Numpy where
print("\nUsing np.where (Sales > 1500: 'High', else: 'Low'):")
print(np.where(df_complex['Sales'] > 1500, 'High', 'Low'))

# Clip values
print("\nClipping values between 1000 and 2000:")
print(df_complex['Sales'].clip(1000, 2000))

# Round values
print("\nRounding to nearest 100:")
print(df_complex['Sales'].round(-2))

```

Summary

This module has covered all the essential topics for working with Pandas DataFrames:

Key Topics Covered:

1. **Introduction to DataFrames** - Understanding the structure and basic properties
2. **Creating DataFrames** - From various sources (dictionaries, lists, arrays, etc.)
3. **DataFrame Attributes and Methods** - Properties, information, and basic operations
4. **Indexing and Selection** - Using loc, iloc, at, iat for data access
5. **Filtering and Boolean Indexing** - Advanced filtering techniques and conditions
6. **Adding and Removing Columns/Rows** - Data manipulation and restructuring
7. **Handling Missing Values** - Detection, filling, and dropping missing data
8. **Data Type Conversion** - Converting between different data types
9. **Sorting and Ranking** - Organizing data and creating rankings
10. **Basic Aggregation Functions** - Statistical operations and grouping
11. **DataFrame Operations and Arithmetic** - Mathematical operations and comparisons

Key Skills Developed:

- Creating and manipulating DataFrames
- Selecting and filtering data effectively
- Cleaning and preprocessing data
- Performing statistical analysis
- Working with missing values
- Converting data types appropriately
- Sorting and ranking data
- Performing aggregation operations

- Applying mathematical operations

Next Steps:

- Practice with real-world datasets
- Explore advanced pandas features
- Learn about data visualization (Module 7)
- Apply these skills to machine learning projects (Modules 8-9)

In []: